L02: The Mathematical Building Blocks of Neural Networks

CSci 560 Deep Learning w/ Python (Chollet) Ch. 2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L02.1 A First Look at a Neural Network

CSci 560: Deep Learning w/ Python (Chollet) Ch. 2.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L02.2 Data Representations for Neural Networks

CSci 560: Deep Learning w/ Python (Chollet) Ch. 2.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L02.3 The Gears of Neural Networks: Tensor Operations

CSci 560: Deep Learning w/ Python (Chollet) Ch. 2.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Matrix · Matrix Dot-Product

- `x`, `y` are input matrices of shape `(a,b)` and `(b, c)` respectively
- Each row of `x` performs an inner product with corresponding column of `y`
- So resulting element `z[i,j]` is the inner product between row `i` of `x` and column `j` of `y`
- Resulting shape of `z` is `(a,c)`

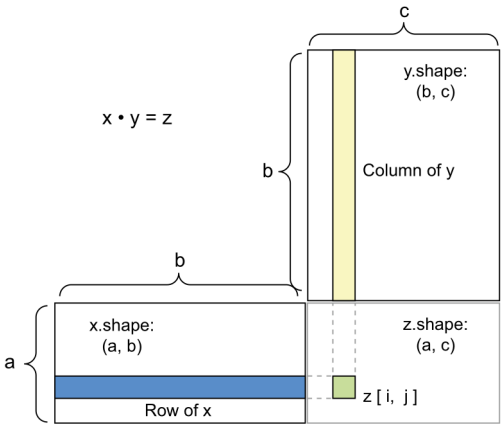


Figure 1: Matrix dot-product box diagram. (Chollet, 2021, pg.43)

Vector Addition is Translation

- Adding a vector to a point will move the point by a fixed amount.
- Applied to a set of points, result is a translation.

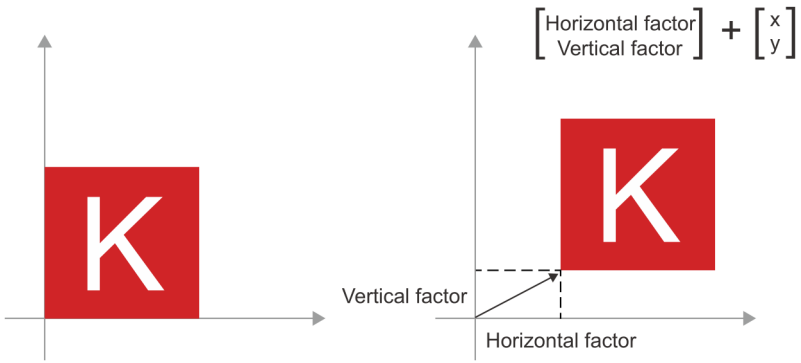


Figure 2: 2D translation as vector addition. (Chollet, 2021, pg.45)

EAST TEXAS A&M

Matrix Multiplication (dot product) is Rotation

- A counterclockwise rotation by angle `theta` achieved by dot product.

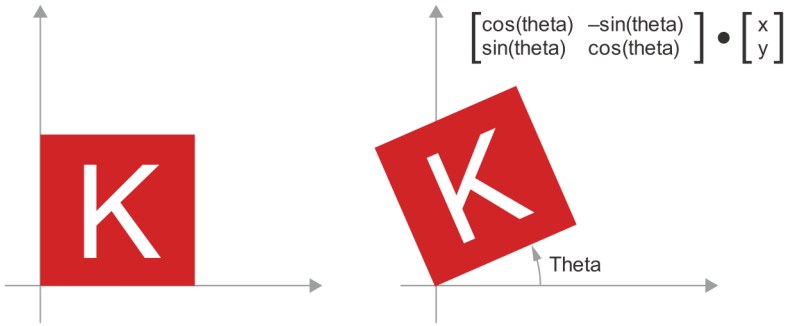


Figure 3: 2D rotation (counterclockwise) as a dot product. (Chollet, 2021, pg.45)

EAST TEXAS A&M

Matrix Multiplication can also Scale

- Vertical and horizontal scaling also achieved by dot product with a suitable matrix.

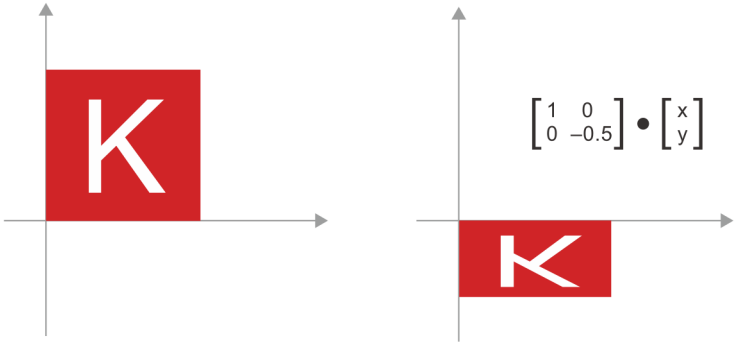


Figure 4: 2D scaling as a dot product. (Chollet, 2021, pg.46)

Affine Transformation: Linear transform and translation

- Translation, Rotation, Scaling are all linear transformations.
- Can combine, for example $y = W \cdot x + b$ is a linear transformation plus a translation.

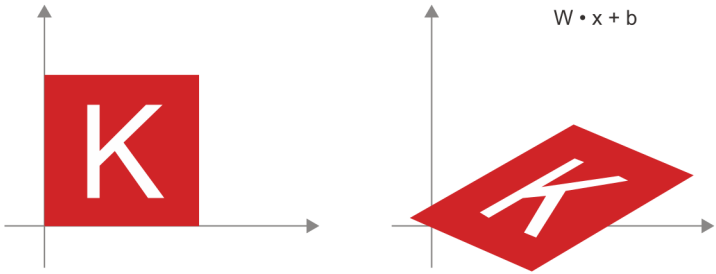


Figure 5: Affine transform in the plane. (Chollet, 2021, pg.46)

EAST TEXAS A&M

`relu` Nonlinear Activation Function

- The previous are just combinations of linear transformations.
- Multilayer NN made entirely of linear transformations can be reduced to a single linear transformation.
- Activation functions produce a nonlinear translation.

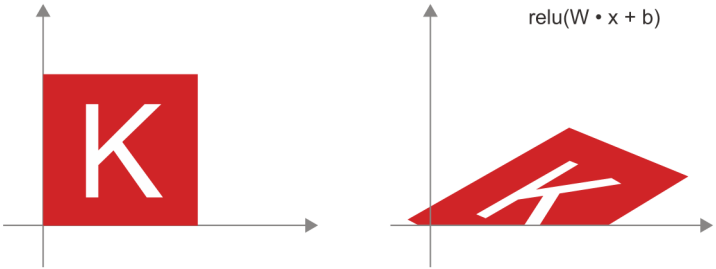


Figure 6: Affine transform followed by (nonlinear) ‘relu‘ activation. (Chollet, 2021, pg.47)

A Geometric Interpretation of Deep Learning

- NN consist entirely of chains of tensor operations.
- Can be interpreted as geometric transformations of the input data into a new space/shape/manifold.
- Can interpret a NN as a very complex geometric transformation in a high-dimensional space, implemented via a series of simple linear + nonlinear transformations.

Figure 7: Uncrumpling a complicated manifold of data. (Chollet, 2021, pg.47)

EAST TEXAS A&M

L02.4 The Engine of Neural Networks: Gradient-based Optimization

CSci 560: Deep Learning w/ Python (Chollet) Ch. 2.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

1D Mini-Batch Stochastic Gradient Descent

- Stochastic because each batch of data is drawn at random.
- Nead a resonable value for the `learning_rate`.
  - too small, slow convergence
  - too large, diverges

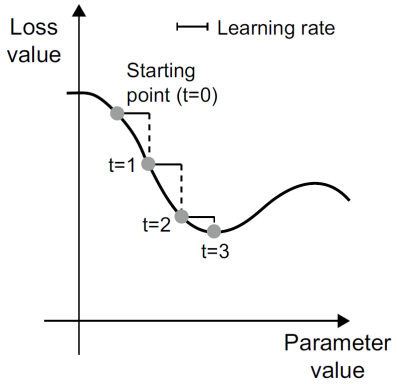
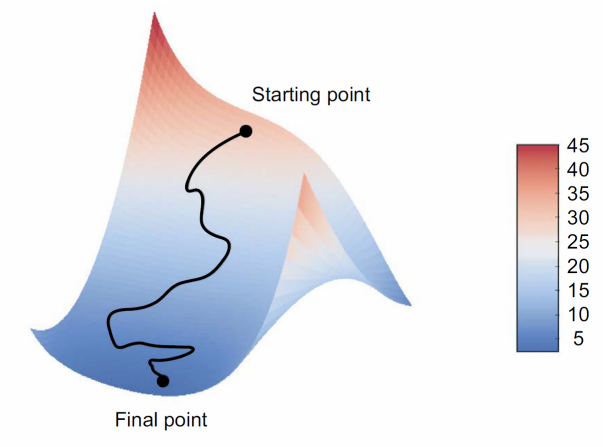


Figure 8: SGD down a 1D loss curve (one learnable parameter) (Chollet, 2021, pg.53)

2D Mini-Batch Stochastic Gradient Descent



- Gradient descent will be used in highly dimensional spaces.
- Every weight coefficient in a neural network is a free dimension in the space.
  - Tens of thousands or even millions of parameters.
- Cannot visualize, and intuitions from low-dimension examples can sometimes be misleading.

Figure 9: Gradient descent down a 2D loss surface (two learnable parameters). (Chollet, 2021, pg.54)

EAST TEXAS A&M

SGD with Momemtum / Advanced Optimization Methods

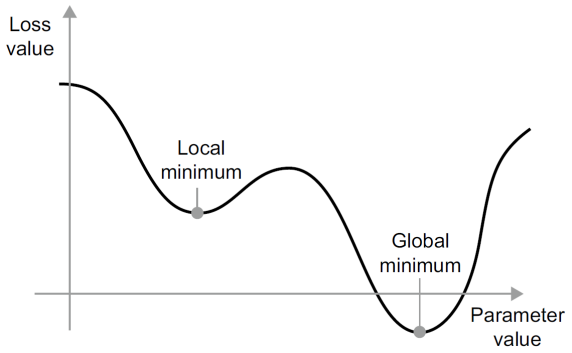


Figure 10: A local minimum and a global minimum. (Chollet, 2021, pg.54)

- **Momentum** take into account previous weight updates rather than just looking at current gradient.
- SGD w/ momentum, Adagrad, RMSprop
- Momentum addresses two issues with SGD: convergence speed and local minima.
  - If enough momentum won't get stuck in local minimum
  - Will adjust learning rate to to make sure convergence does not take too long.
- Implemented by moving each step based on both slope (current acceleration) and velocity.

EAST TEXAS A&M

Chaining Derivatives: The Backpropagation Algorithm

- **Backpropagation** is a way to use the derivatives of simple operations to easily compute the gradient of arbitrarily complex combinations of atomic operations.
- A NN consists of many tensor operations chained together, each of which has a simple known derivative.
- The chain rule states that
  `grad(y, x) == grad(y, x1) * grad(x1, x)`
- Can be applied to any number of imtermediate functions

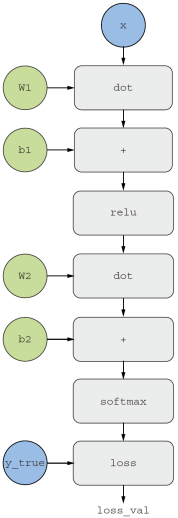
```
grad(y, x) == (grad(y, x3)  * grad(x3, x2) *
               grad(x2, x1) * grad(x1, x))
```

```
def fg(x):
    x1 = g(x)
    y = f(x1)
    return y

def fghj(x):
    x1 = j(x)
    x2 = h(x1)
    x3 = g(x2)
    y = f(x3)
    return y
```

EAST TEXAS A&M

Computation Graphs and Autometic Differentiation



- Useful way to think of and perform backpropagation: use computation graphs.
- A **computation graph** is a directed acyclic graph of operations.
- Treat computation as data:
  - Graph can be built in `TensorFlow` from a `GradientTape`
  - Represents forward pass
  - Can reverse links to perform backpropagation backward pass.

Figure 11: The computation graph

Automatic Differentiation: Forward Pass

- Consider a simplified computation graph.
  - single layer all variables are scalar
  - scalar `w` for weights and `b` for bias
  - scalar `x` input
  - no activation function
  - loss is difference of result and `y_true`
- We want to update `w` and `b` in a way that will minimize `loss_val`.
  - want to compute `grad(loss_val, b)` and ‘grad(loss_val, w).
- Propagate these values to all nodes in the graph from top to bottom, until we reach `loss_val`.
  - This is the **forward pass**

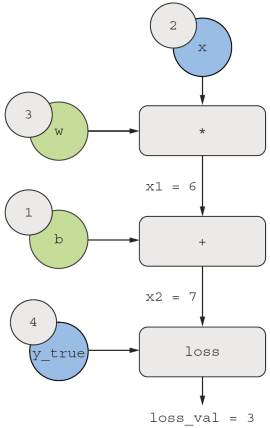


Figure 12: Running a forward pass. (Chollet, 2021, pg.58)

Automatic Differentiation: Backward Pass / Backpropagation

- "reverse" the graph
  - For each edge from `A` to `B`, create opposite edge from `B` to `A`
  - backward graph represents the **backward pass**, a concrete implementation of performing **backpropagation**
  - You can obtain the derivative of a node with respect to another node by *multiplying the derivatives for each edge along the path linking the two nodes*.

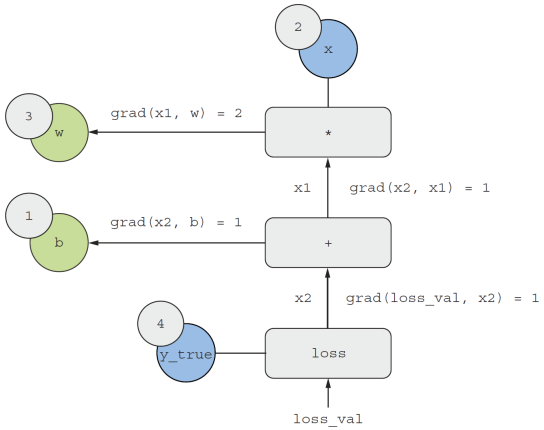


Figure 13: Running a forward pass. (Chollet, 2021, pg.58)

Automatic Differentiation using the Computation Graph

- Chain rule says the backword graph can be used to obtain the derivatives of the learnable parameters with respect to the loss.
- **Multiply the derivitives for each edge along the path linking two nodes**.
- For instance:

```
grad(loss_val, w) =
    grad(loss_val, x2) *
    grad(x2, x1) *
    grad(x1, w)
```

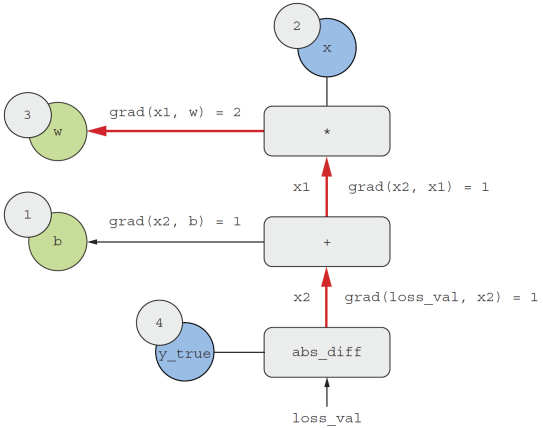


Figure 14: Path from loss_value to w in the backward graph. (Chollet, 2021, pg.59)

L02.5 Looking Back at our First Neural Network Example

CSci 560: Deep Learning w/ Python (Chollet) Ch. 2.5

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Bibliography

Chollet, F. (2021). *Deep learning with python* (second). Manning.

EAST TEXAS A&M